Wirtualna i rozszerzona rzeczywistość

Wykład 6

Renderowanie i wydajność aplikacji XR

dr inż. Mateusz Pomianek

Automatyka i Robotyka, studia II stopnia, 3 ECTS

Katedra Informatyki i Automatyki

POLITECHNIKA RZESZOWSKA im. IGNACEGO ŁUKASIEWICZA

KATEDRA INFORMATYKI I AUTOMATYKI POLITECHNIKI RZESZOWSKIEJ

Rok akademicki 2026/2027

Wersja 1.5.2 14.10.2026

Plan i rytm wykładu

1. Punkt wyjścia: działająca aplikacja
2. Stos XR i budżet klatki
3. Demonstracja: odczyt profilu
4. Przerwa
5. Optymalizacja i testowanie
6. Demonstracja: porównanie profili
7. Wynik: procedura diagnostyczna

Jedna przerwa rozdziela architekturę stosu XR od optymalizacji i testowania.

Po tym wykładzie potrafisz...

- odczytać budżet klatki i rozróżnić obciążenie CPU i GPU,
- uruchomić profilowanie tej samej sceny na urządzeniu docelowym,
- dobrać LOD, culling albo batching do wskazanego wąskiego gardła,
- zaprojektować wiarygodne porównanie pomiarów przed i po optymalizacji.

Jedna scena Unity, dwa środowiska pomiaru

Edytor na komputerze

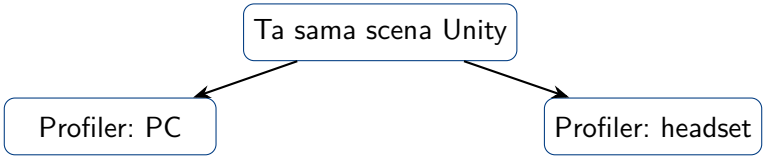
Unity Play Mode

CPU, GPU i widok gry w środowisku pracy edytora.

Urządzenie docelowe

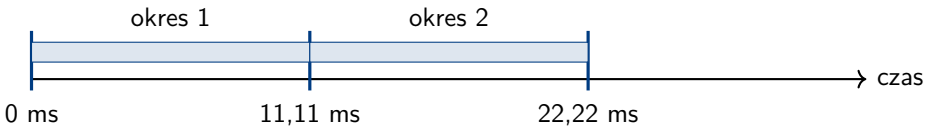
Samodzielny headset XR

Osobny procesor, GPU, sterowniki i ograniczenia termiczne.



90 Hz oznacza nową klatkę co 11,11 ms

$$T = \frac{1000\ \text{ms}}{90} \approx 11{,}11\ \text{ms}$$

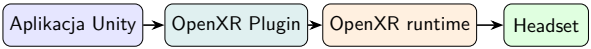


Co pokaże Profiler?

Obciążenie CPU i GPU trzeba zmierzyć osobno. Odstęp między klatkami nie jest tożsamy z czasem pracy samego GPU ani z opóźnieniem motion-to-photon.

OpenXR: wspólny standard zamiast wielu API

Różne interfejsy sprzętowe zwiększały koszt integracji. **OpenXR** standaryzuje komunikację aplikacji z runtime urządzenia, model sesji, wejścia i wyświetlania.



Unity, XRI i AR Foundation: trzy różne warstwy

- **OpenXR Plugin**: komunikacja Unity z runtime urządzenia XR.
- **XR Interaction Toolkit**: gotowe mechanizmy chwytania, promieni, teleportacji i UI przestrzennego.
- **AR Foundation**: wspólna warstwa dla funkcji AR, takich jak płaszczyzny, kotwice i śledzenie środowiska.

Ważne

Te warstwy rozwiązują inne problemy. Nie należy traktować ich jako zamienników.

Cykl sesji XR



Praktyczna konsekwencja

Aplikacja może być widoczna, ale nie mieć fokusu. W takim stanie nadal renderuje scenę, lecz nie powinna przyjmować wejścia użytkownika jak w stanie FOCUSED.

Rendering stereoskopowy: dwa oczy, dwa przebiegi?

Multi-pass

- osobny przebieg dla lewego oka,
- osobny przebieg dla prawego oka,
- większy koszt CPU i draw calls.

Single Pass Instanced

- obie perspektywy w jednym przebiegu,
- wykorzystanie instancjonowania GPU,
- niższy narzut CPU dla tej samej sceny.

Budżet klatki: liczba, której nie można ignorować

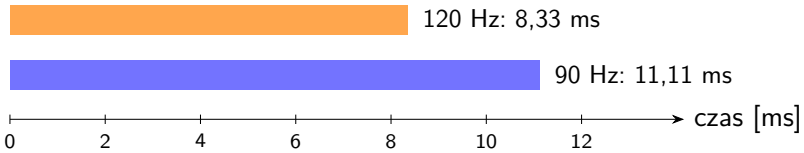
11,1 ms
przy 90 Hz

8,3 ms
przy 120 Hz

± 2 **ms**
cel dla stabilności czasu klatki

Związek z opóźnieniem

Gdy renderer nie mieści się w budżecie, pojawia się stara klatka, reprojekcja albo judder. Przekroczenie budżetu jest jedną z przyczyn wzrostu motion-to-photon latency.



Renderowanie skupione (dołkowe)

Nazwa metody pochodzi od **dołka środkowego siatkówki** (łac. *fovea*), czyli obszaru widzenia centralnego o największej ostrości.

- centralna część pola widzenia jest percepcyjnie najważniejsza,
- peryferia mogą być renderowane w niższej rozdzielczości,
- **stałe renderowanie skupione** (fixed foveated rendering) utrzymuje obszar najwyższej jakości w ustalonym miejscu,
- **renderowanie skupione śledzące wzrok** (eye-tracked foveated rendering) przesuwa ten obszar zgodnie ze spojrzeniem.

Połączenie z Wykładem 2 i 5

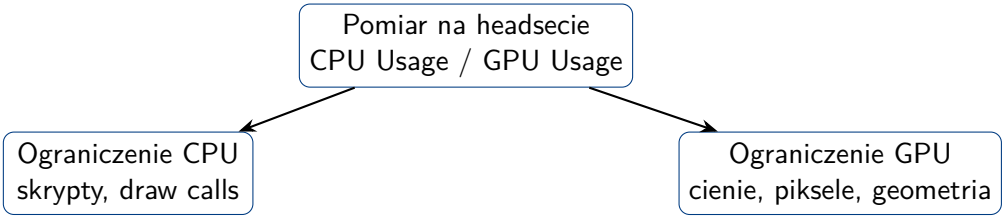
Optymalizacja GPU opiera się tu bezpośrednio na fizjologii widzenia, PPD i pomiarze kierunku spojrzenia.

Kod C#: kontrolowanie budżetu klatki

1. `Time.unscaledDeltaTime` daje czas między wywołaniami klatek.
2. `1000 / targetRefreshHz` wyznacza orientacyjny budżet w ms.
3. Zliczaj przekroczenia i raportuj je okresowo, nie w każdej klatce.
4. Czas CPU i GPU badaj osobno w **Unity Profiler**.

Pełny, wykonywalny przykład: `snippets/frame_budget.cs`.

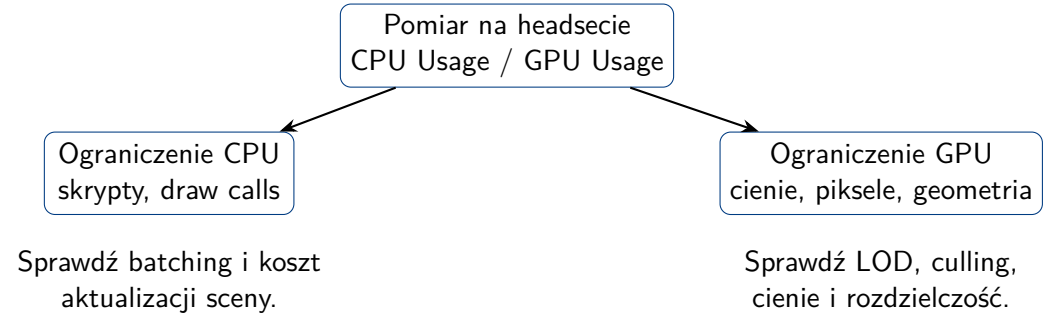
Unity Profiler: najpierw rozpoznaj ograniczenie



Weryfikacja

Po jednej zmianie uruchom ponowny pomiar na **tym samym urządzeniu**, tej samej scenie i w porównywalnych warunkach.

Unity Profiler: najpierw rozpoznaj ograniczenie



Weryfikacja

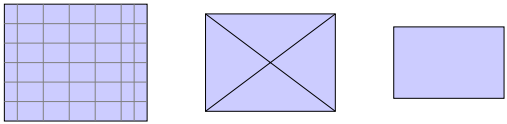
Po jednej zmianie uruchom ponowny pomiar na **tym samym urządzeniu**, tej samej scenie i w porównywalnych warunkach.

Przerwa 5 min

LOD: mniej geometrii tam, gdzie jej nie widać

LOD: odleglejsze obiekty używają modeli o mniejszej liczbie trójkątów. Optymalizacja jest skuteczna, gdy koszt geometrii rzeczywiście ogranicza GPU.

Uproszczone modele ilustracyjne



Culling: nie renderuj tego, czego nie zobaczysz

Frustum culling

Obiekty poza bryłą widzenia kamery nie trafiają do renderowania.

Occlusion culling

Obiekty całkowicie zasłonięte przez inne obiekty mogą zostać pominięte.

Nie mylić z okluzją AR

Occlusion culling optymalizuje wydajność. Okluzja z Wykładu 4 odpowiada za poprawność relacji głębi między światem realnym i wirtualnym.

Batching i instancing: mniej poleceń, ta sama scena

- każdy draw call ma koszt po stronie CPU,
- tysiąc małych obiektów może być droższy niż kilka dużych,
- batching łączy zgodne obiekty w mniej wywołań,
- GPU instancing pozwala rysować wiele kopii tej samej geometrii jednym mechanizmem.

Przykład XR

Duża chmura punktów lub wiele identycznych znaczników może być ograniczona przez liczbę wywołań, nawet jeśli pojedynczy element jest bardzo prosty.

Profiler: najpierw diagnoza, potem optymalizacja

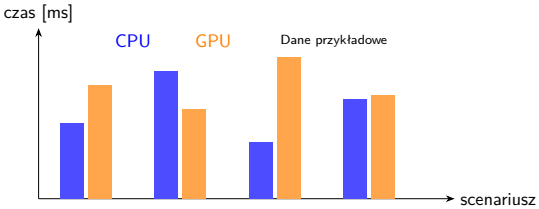
CPU-bound

- dużo draw calls,
- logika skryptów,
- synchronizacja i przetwarzanie danych.

GPU-bound

- kosztowne shadery,
- wysoka rozdzielczość,
- zbyt dużo geometrii lub overdraw.

Profilujemy na urządzeniu docelowym, nie tylko w edytorze.

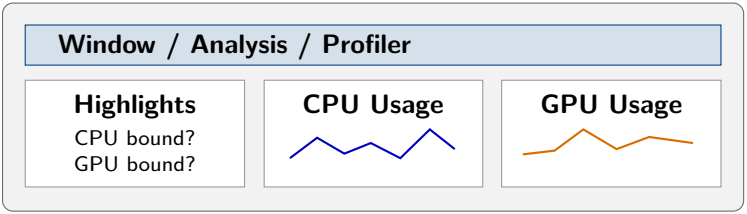


Testowanie: wydajność to także ludzie

- testuj na rzeczywistych urządzeniach docelowych,
- kontroluj nie tylko średni FPS, lecz także piki czasu klatki i jitter,
- uwzględnij różne scenariusze obciążenia,
- testuj komfort z różnymi użytkownikami,
- wyniki liczbowe łącz z obserwacją objawów i czasem wykonania zadań.

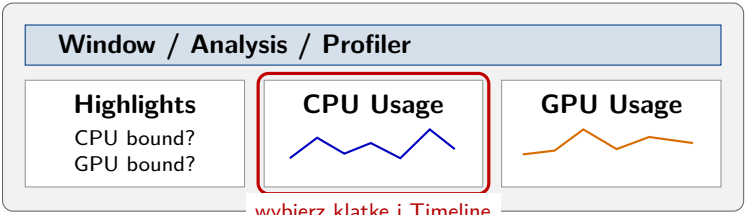
Wersjonowanie: assety XR są częścią oprogramowania

- duże modele, tekstury i nagrane chmury punktów przechowuj przez Git LFS,
- konfiguracje jakości i build targets trzymaj w repozytorium,
- nie przełączaj ręcznie ustawień platformowych przed każdym buildem,
- wersjonuj także parametry, które wpływają na jakość i wydajność.



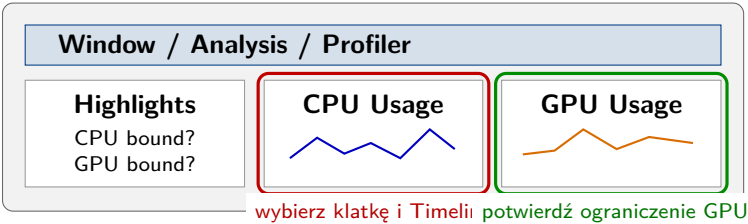
Jak interpretować

Dla 90 Hz budżet okresu klatki wynosi $1000/90 \approx 11{,}11$ ms. Rzeczywiste koszty CPU i GPU odczytuje się z zapisanych profili urządzenia.



Jak interpretować

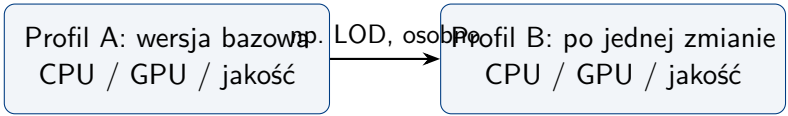
Dla 90 Hz budżet okresu klatki wynosi $1000/90 \approx 11{,}11$ ms. Rzeczywiste koszty CPU i GPU odczytuje się z zapisanych profili urządzenia.



Jak interpretować

Dla 90 Hz budżet okresu klatki wynosi $1000/90 \approx 11{,}11$ ms. Rzeczywiste koszty CPU i GPU odczytuje się z zapisanych profili urządzenia.

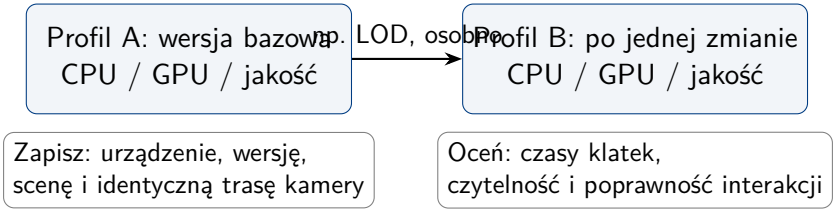
To samo urządzenie, ta sama scena, jedna zmiana



Dowód poprawy

Potrzebne są dwa rzeczywiste profile z tej samej platformy. Sam schemat nie dowodzi przyspieszenia.

To samo urządzenie, ta sama scena, jedna zmiana



Dowód poprawy

Potrzebne są dwa rzeczywiste profile z tej samej platformy. Sam schemat nie dowodzi przyspieszenia.

Kod C#: zapis czasu Main Thread

```
1   ProfilerRecorder mainThread;
2
3   void OnEnable()
4   {
5       mainThread = ProfilerRecorder.StartNew(
6           ProfilerCategory.Internal, "Main Thread", 60)
     ;
7   }
8
9   void Update()
10  {
11      if (mainThread.Valid && Time.frameCount % 60 ==
     0)
12          Debug.Log($"Main Thread: {mainThread.
     LastValue / 1e6:F2} ms");
13  }
```

API Unity

`ProfilerRecorder`: próbki `Main Thread`.

Jednostka

10^6 ns $= 1$ ms.

Uwaga

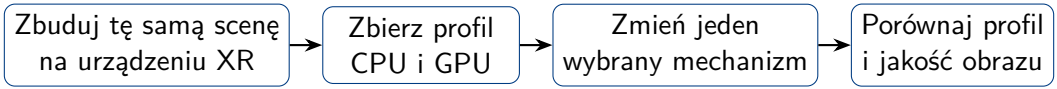
Pomiar jednego wątku, nie całej klatki.

Zrzut: rzeczywisty profil CPU/GPU na urządzeniu XR



Miejsce na rzeczywisty zrzut ekranu: Unity Profiler po podłączeniu rzeczywistego headsetu: nazwa urządzenia, CPU Usage / Timeline, wybrana klatka, GPU Usage, jeżeli licznik jest obsługiwany.

Decyzja o optymalizacji wynika z pomiaru



Dowód poprawy

Spadek rzeczywistego kosztu ograniczającego, mierzony w porównywalnych warunkach.

Warunek techniczny

Brak utraty poprawności interakcji, geometrii i czytelności sceny.

Przykładowe pytania egzaminacyjne

1. Jak wyznaczyć budżet czasu pojedynczej klatki dla 90 Hz i odróżnić go od opóźnienia całego toru XR?
2. Jak przy użyciu Unity Profiler rozstrzygnąć, czy scenę ogranicza CPU, GPU czy synchronizacja?
3. Jakie problemy wydajności rozwiązują odpowiednio LOD, culling i batching?
4. Jak zorganizować porównywalny pomiar wydajności sceny Unity przed optymalizacją i po niej?

Bibliografia: kluczowe źródła

XR i rendering

- Khronos Group, *OpenXR Specification*.
- Unity Technologies, *Unity Manual: XR*.
- S. M. LaValle, *Virtual Reality*, 2017.

Optymalizacja

- Unity Technologies, *Unity Profiler Manual*.
- Unity Technologies, *Graphics performance fundamentals*.
- Khronos Group, *OpenXR SDK and API reference*.

Pełna bibliografia: `../../bibliography.bib`.

Kolejne materiały

← Wykład 5	Laboratorium 6	Wykład 7 →